

AnyGrasp: Robust and Efficient Grasp Perception in Spatial and Temporal Domains

Hao-Shu Fang, Chenxi Wang, Hongjie Fang, Minghao Gou,
Jirong Liu, Hengxu Yan, Wenhai Liu, Yichen Xie, Cewu Lu, *Member, IEEE*

Abstract—As the basis for prehensile manipulation, it is vital to enable robots to grasp as robustly as humans. In daily manipulation, our grasping system is prompt, accurate, flexible and continuous across spatial and temporal domains. Few existing methods cover all these properties for robot grasping. In this paper, we propose a new methodology for grasp perception to enable robots these abilities. Specifically, we develop a dense supervision strategy with real perception and analytic labels in the spatial-temporal domain. Additional awareness of objects' center-of-mass is incorporated into the learning process to help improve grasping stability. Utilization of grasp correspondence across observations enables dynamic grasp tracking. Our model, AnyGrasp, can generate accurate, full-DoF, dense and temporally-smooth grasp poses efficiently, and works robustly against large depth sensing noise. Embedded with AnyGrasp, we achieve a 93.3% success rate when clearing bins with over 300 unseen objects, which is comparable with human subjects under controlled conditions. Over 900 MPPH is reported on a single-arm system. For dynamic grasping, we demonstrate catching swimming robot fish in the water.

Index Terms—General grasping, dynamic grasping, AnyGrasp

I. INTRODUCTION

VISUAL guided object grasping is important in the robotic community. Other than solely picking objects in the industrial environment, we would foresee robots working closely with humans even in household environments. Thus, a grasping system that can serve most daily manipulation in unstructured environments is attractive.

To achieve that, we first introspect our own grasping ability. For humans, we perceive a partial observation without a full model of the scene. Our visual system processes such input within 100ms and we would then know how to pre-shape our hand for comforting contact during grasping [1]. Such a grasp system is accurate and robust for any object. Moreover, a time-consistent property is guaranteed so that we can recognize and grasp moving objects in dynamic scenes.

To date, researchers have proposed different methods for visual-guided grasping. Some methods [2], [3] assumed full knowledge of the object and contact model, which does not always hold in the real world. Some simplified the grasping perception as a planar detection problem [4], [5] but would impose restrictions on afterward manipulation. Some proposed sampling-evaluation methodologies that first sample

candidates on the scene and then evaluate their quality [5]–[8], which is time-consuming and cannot generate dense predictions. And these methods mainly focused on static scene grasp detection. The dynamic grasp detection problem remains fewly explored.

In this paper, we present a spatially and temporally unified methodology, AnyGrasp, to bridge the grasp perception ability gap between robots and humans. Specifically, a geometry processing module estimates dense 6DoF grasp poses for a monocular perceived observation in one feed-forward pass. A temporal association module identifies the grasp correspondence among these tremendous grasp poses across every two observations. To reduce the burden of explicit collision detection during grasping, our model is obstacle-aware and eliminates the grasp candidates that have no space for hand placing. This helps accelerate the selection of grasp poses in afterward manipulation. To improve stability, the awareness of the center of gravity (COG) for objects is also equipped. These two properties are also witnessed in human's visually guided grasping behavior [1]. Our model can generate accurate, full-DoF and continuous grasp poses across space and time in 100ms.

A problem during learning is that we lack a dataset for dynamic object grasping in the real world. Previous grasp learning resorts to collecting data in simulation. However, we show in this paper that when using a low-cost commercial depth sensor, an algorithm with simple sim-to-real transferring techniques still performs worse than directly training with real-world data. Thus, we persist in training with real data. To avoid extensive human labor, we exploit the grasp pose correspondence across different observations of a static scene and propose a dense supervision strategy with real perception and analytic labels in the spatial-temporal domain that greatly improves data efficiency.

We have conducted relevant research [9]–[11] before. However, we only considered the grasping problem in static scenes and did not fully evaluate the grasp perception system in different scenarios. Thus, in this paper, we develop the grasping ability in dynamic scenes that is complementary to our previous work. And we extensively evaluate the grasping system in a variety of tasks. Extra analyses of different principles in dataset design are also conducted.

Specifically, we first test our algorithm in a large-scale bin-picking experiment with over 300 unseen objects with diverse shapes, materials, and sizes. On average, a success rate of over 93% is reported, which is on par with the human success rate when the human subjects adopt the same configuration of

H-S. Fang, C. Wang, H. Fang, M. Gou, J. Liu, H. Yan, W. Liu, Y. Xie and C. Lu are with the School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University, Shanghai, 200240, China. C.Lu is the corresponding author. Authors' email: fhaoshu@gmail.com, {wxc1997, galaxies, gmh2015, jirong, hengxuyan, sjtu-wenhai, xieyichen, lucewu}@sjtu.edu.cn

end-effector and an open-loop grasping strategy. We further carry out experiments of our method on different depth sensing noise to show its robustness. For dynamic scenes, the robot successfully demonstrates catching moving robot fish in the fish tank, which is difficult due to the small friction in the water and the tiny size of the fish. Finally, we discuss the influence of several factors in dataset construction, such as real-world data versus simulated data, the number of grasp annotations on an object, the scene diversity, etc. These analyses may be helpful for future data collection in this area.

To support reproducible research, the library and example code of AnyGrasp system is provided at <https://graspnet.net/anygrasp.html>.

II. RELATED WORK

In this section, we introduce some background material that most relates to the visual grasping topic. The scope is restricted to the two-finger parallel gripper, which we mainly discuss in this paper. This section is divided into two parts, where Sec. II-A focuses on grasp pose detection methods and Sec. II-B focuses on continuous action learning methods.

A. Grasp Pose Detection

The grasp pose detection problem can be defined as predicting several poses for a given scene, usually in the Cartesian space, such that the robot can successfully lift the objects when moving its end-effector to those poses. Early methods would assume full 2D or 3D knowledge of the objects [12], [13], or approximate the objects into a set of primitive shapes [14], [15]. These methods would meet limitations in real-world environments where the 3D models of objects are hard to obtain. Learning-based methods help to alleviate this dilemma through large-scale data and automatic feature extraction. Representations for grasp pose detection evolved with the development of learning-based methods. Saxena *et al.* [16]–[18] proposed a point-based representation, where a single point indicates the grasp pose location. Later, Le *et al.* [19] proposed a point pair representation that is easier to learn. In [20], Jiang *et al.* used a rectangle to represent the grasp pose, which took the physical size of the gripper into consideration. This rectangle-based representation, as well as the Cornell dataset [20], [21], became widely adopted in planar grasp pose detection. Later, Mahler *et al.* [5], [22] concluded two concerns regarding the human-labeled datasets that it may require significant data collection time, and the grasp quality annotation is not guaranteed. Thus, they proposed to learn with simulated data and robust analytic grasp metrics. Morrison *et al.* [23] focused on another aspect of the grasping problem that previous methods only generated sparse grasp poses for a scene. They developed GGCNN, which generates pixel-wise grasp quality maps based on the rectangle annotation in [21].

The above methods mainly generate 4 DoF grasp poses on the camera plane. The limitation in degree of freedom may neglect some vital grasp poses, *e.g.* grasp poses on the edge of a plate, and result in failure. To generate the full 6 DoF grasp poses, Ten Pas *et al.* [6] proposed a sampling-evaluation-based method that firstly samples grasp candidates

on point cloud and then evaluates their grasp qualities with a neural network. Mousavian *et al.* [8] generated grasp candidates with a variational autoencoder and refined the grasp poses iteratively. A major drawback of the sampling-evaluation methods is that they need to trade off the computation time and the number of generated grasp poses. Thus, they usually took several seconds to run and only generated tens of grasp poses for a scene. Recently, Fang *et al.* [9] proposed an end-to-end network that directly generates abundant grasp poses for an input scene point cloud. For different grasp points densely sampled in the scene, they regressed the grasp quality scores for poses in different viewpoints and in-plane rotations. A large-scale dataset with real data and analytic labels was also built. Similar networks were parallelly developed in [24], [25]. Wang *et al.* [11] proposed a graspsness module that eliminates unfeasible grasp poses in the early stage of the network, which greatly improves the grasp success rate. Other than point cloud, some researchers also explored other representations of a scene, including RGB/D image [10], [26] and neural radiance field [27]. A recent survey [28] provided a full summary of the 6 DoF grasping methods. In this paper, we follow the end-to-end methodology and mainly build our network upon [11].

The problem formulation of grasp pose detection constrains them to focus on static scenes. In order to extend to the dynamic scene, classic methods usually require prior information about the objects [29]–[31], or operate on a fixed set of grasp trajectories [32]. For dynamic grasping without prior, existing methods either choose the nearest grasp pose in the next image as the same grasp target [23] or generate possible future candidates and evaluate their grasp quality [33]. Both methods only guarantee the tracked grasp pose has a small distance with respect to the last frame in the image coordinate system, but cannot guarantee a small distance in the object coordinate system. In this paper, we propose a new generation-association methodology for dynamic grasp pose detection.

B. Continuous Action Learning for Grasping

Different from the above methods that detect the grasp poses in the scene and move toward the target grasp pose using a motion planner, this line of research directly maps the observation to a continuous action space. We summarize some representative works. Levine *et al.* [34] collected 800K grasp attempts on real robots and learn a CNN grasp predictor that associates image and motor command. Viereck *et al.* [35] proposed to learn with simulated point clouds, such that no real-robot trials are needed. Bousmalis *et al.* [36] investigated the sim-to-real transfer problem in grasp action learning. Song *et al.* [37] collected grasping demonstrations with a low-cost hand-held gripper. Wang *et al.* [38] extended the continuous action space to 6D. A problem of these methods is that they may be prone to the domain shift problem [39], [40] and perform less robust than the detection-based methods.

III. ANYGRASP DESIGN PRINCIPLES

In this section, we first describe the problem definition of our spatial-temporal grasp detection. Then we introduce the principles of our system design, from both the algorithm and data perspectives.

A. Problem Definition

For the parallel jaw based grasping, we represent a grasp pose $\mathcal{G}$ as

$$\mathcal{G} = [\mathbf{R} \ \mathbf{t} \ w], \quad (1)$$

where $\mathbf{R} \in \mathbb{R}^{3 \times 3}$ denotes the gripper orientation, $\mathbf{t} \in \mathbb{R}^{3 \times 1}$ denotes the center of grasp and $w \in \mathbb{R}$ denotes the minimum gripper width that is suitable for grasping the target object. Let $\mathcal{E}$ denotes the environment including robot and objects, $\mathcal{P}$ denotes the partial-view point cloud from a depth camera, and $s(\mathcal{E}, \mathcal{P}, \mathcal{G})$ denotes a binary variable indicating grasp $\mathcal{G}$ success or not given environment $\mathcal{E}$ and perception $\mathcal{P}$. A grasp is successful if the object is lifted successfully. Our goal is to find a set of grasp poses $\mathbf{G} = \{\mathcal{G}_1, \mathcal{G}_2, \dots, \mathcal{G}_n\}$ that maximizes the grasp success rate given a fixed n :

$$\{\mathcal{G}_1^*, \mathcal{G}_2^*, \dots, \mathcal{G}_n^*\} = \arg \max_{|\mathbf{G}|=n} \sum_{\mathcal{G}_i \in \mathbf{G}} \text{Prob}(s = 1 | \mathcal{E}, \mathcal{P}, \mathcal{G}_i). \quad (2)$$

This means that we hope our algorithm predicts abundant grasp poses to cover the whole scene so that we can have different candidates for grasp execution.

When introducing the temporal dimension, we denote $\mathcal{E}^j, \mathcal{P}^j, \mathbf{G}^j$ as the environment, perception and grasp poses at time j , $\text{dist}(\mathcal{G}_k^j, \mathcal{G}_k^{j-1} | \mathcal{E}^j, \mathcal{E}^{j-1})$ as the distance of the paired grasp poses across two moments under the coordinate system of the grasped target. Thus, we would have:

$$\begin{aligned} \{\mathcal{G}_1^*, \mathcal{G}_2^*, \dots, \mathcal{G}_n^*\}^{(j)} &= \arg \max_{|\mathbf{G}^j|=n} \sum_{\mathcal{G}_i \in \mathbf{G}^j} \text{Prob}(s = 1 | \mathcal{E}^j, \mathcal{P}^j, \mathcal{G}_i), \\ \text{s.t. } \text{dist}(\mathcal{G}_k^j, \mathcal{G}_k^{j-1} | \mathcal{E}^j, \mathcal{E}^{j-1}) &\leq \delta, \forall \mathcal{G}_k^{j-1} \in \mathbf{G}^{j-1}, \end{aligned} \quad (3)$$

δ is the tolerance error, under which two grasp poses could be regarded as the same pose.

B. Spatial-Continuous Learning

Previous methods [6], [7] adopt the sampling strategy that chooses candidates on point cloud and evaluates their quality. Instead, our geometry processing module directly perceives the single-view point cloud of the scene and estimates the grasp quality across the $\mathbb{R}^6$ space. We refer to it as “spatial-continuous learning” owing to the dense property of the predicted grasp poses, from which we can query a feasible grasp pose at any target location in most cases. Compared to the sampling-based methods that classify the quality of a grasp solely based on the cropped local point cloud within the gripper space, our method also considers the geometric structure from neighbor regions that could provide richer information conveying whether a grasp is of good quality or not. Such global geometric features can be easily learned through a 3D convolutional network. By taking the whole scene as input, two extra advantages are witnessed. Firstly, to improve the stability of grasping, human tends to put visual attention on the center of gravity (COG) of the object at the preparing stage of grasping [1]. Since the mass distribution is unknown, humans usually assume the COG is the center of the object. Such intuition can be modeled by our neural network that directly perceives the whole scene. In our network, we

encode such information for each grasp pose by predicting a normalized vertical distance from the gripper plane to the COG of the grasped object. Secondly, researchers in cognition found that humans would visually attend to the obstacle during the preparation period of grasping [1], [41]. That is, the visual system would also consider the obstacle and avoid collision for hand pre-shaping. Benefiting from the full point cloud observation, we bring such capability to our network. If there is an obstacle around the grasp pose and leaves no space for gripper pre-shaping, the grasp quality score is directly set to zero. Although obstacle avoidance can also be performed using mesh and point cloud, the neural-based implicit collision detection helps eliminate a large portion of unreachable grasp poses and reduces the expensive computation time.

C. Temporal-Continuous Learning

When trying to grasp a moving object, the robot needs to continuously update the target grasp pose before catching the object. The grasp poses between each two frames should have small SE(3) distance in the object’s coordinate system to enable a smooth, target-consistent movement of the robot gripper. We refer to the learning of such property as “temporal-continuous learning”.

Previous sampling-based method [33] added some disturbance to the grasp pose from the previous frame and evaluate these candidates in the current frame’s geometry. To ensure computation efficiency, the generated proposals are usually sparse. Thus they cannot cover all the possible movements of the objects.

To avoid the speed-accuracy tradeoff, we propose a new generation-association methodology to ensure dense and consistent grasp poses temporally. Given two observations at different times, our geometry processing module generates dense grasp poses across the scene. A temporal association module takes the grasp poses and their corresponding geometric features encoded by the spatial model as input and produces their many-to-many association score matrix. The association score between each two grasp poses denotes their consistency in the temporal domain, measured by their SE(3) distance in the grasping object’s coordinate system. Contributed to the spatial continuous property of our geometry processing module, we could generate dense grasp poses around a selected target to ensure both temporal continuity and grasp quality.

D. Training Data

For data collection, most methods for high DoF grasp pose detection directly learn from simulation. The main reason is that obtaining dense grasp pose annotation for data collected in the real world is difficult. However, although turning to simulation lowers the training cost, an expensive and high-precision depth camera is needed [7] to achieve good performance during the inference phase. This is to bridge the sim-to-real gap since simulation produces perfect partial view depth. In contrast, we resort to directly learning with the real perception which requires more effort for data collection, but can allow the algorithm to adapt to real-world noise, especially on low-cost cameras. In our experiments, we show how our

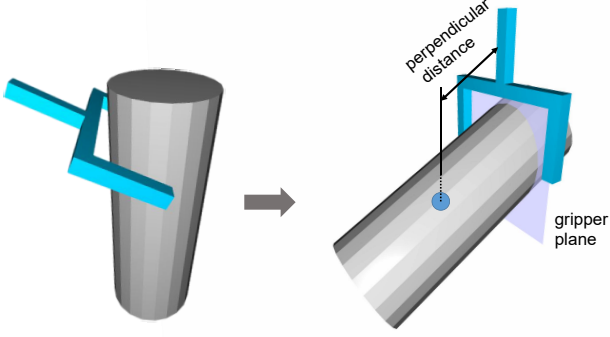


Fig. 1. An illustration of the perpendicular distance from the gripper plane to the COG of the object.

grasp detection system can tolerate different sensing noises, and improve the success rate by a large margin over its counterpart trained in simulation.

IV. METHODS AND MATERIALS

A. Data Collection and Annotation

For the training data of our algorithm, we mainly adopt the training set of GraspNet-1Billion [9] and follow the same methodology to collect 168 extra scenes composed of 104 new objects. In total, 268 scenes that consist of 144 objects are used to train our network. In brief, we obtain objects' 3D mesh models with a commercial 3D scanner and calculate dense grasp poses using analytic antipodal scores [5], [6]. For each scene, we randomly choose several objects (~ 10) and randomly placed them on the table. Images are taken at 256 different viewpoints for each scene and we manually annotate the object 6D poses in the scene. Then the grasp poses on the scene can be obtained by projecting the poses on each object with the annotated 6D pose to the scene. Moreover, collision detection is performed by assuming a voxel size of the gripper and checking whether each grasp pose will have an intersection with the object models in the scene. For more details, we refer readers to [9].

In this paper, we annotate three extra labels for the training data, which help improve the stability during grasping and enable the grasp pose tracking ability.

Firstly, the original grasp pose annotation consists of four approach depths, *i.e.*, 1, 2, 3, and 4 centimeters. In this paper, we add an extra depth of 0.5 centimeters to grasp small objects.

Secondly, as illustrated in [1], human attention will bias to the center of gravity of the object during grasping. Inspired by this, we define a stable score for the grasp pose. We assume that the gripper will lift the object vertically along the grasp pose orientation. Thus, we define the stable score as the normalized perpendicular distance between its gripper plane and the COG of the object. The normalization process is conducted by obtaining all the perpendicular distances for an object and then dividing them by the maximum distance among them. In this case, the lower this score is, the more disturbance the grasp pose can tolerate since the gravity moment is more balanced. To obtain the annotation, we first

compute the COG of the object. Since the center cannot be precisely computed only from visual perception, the object is assumed to be a solid rigid body with uniform density, which is similar to human intuition. Then, the COG is transformed into the gripper coordinate system according to the grasp pose. The perpendicular distance is the distance from the COG to the gripper plane. Fig. 1 gives an example of calculating the perpendicular distance for a grasp pose. Finally, all the distances are collected and normalized to $[0, 1]$ to get stable scores.

Thirdly, to ensure temporal consistency of the grasp poses, our algorithm associates the grasp poses that represent the same pose *w.r.t* the target object between two frames. To achieve that, a training dataset that contains the association label is needed. We generate such association labels for our collected training data. Although the original dataset does not contain dynamic scenes with moving objects, it contains images captured from 256 different viewpoints for each scene. The images captured from adjacent viewpoints present subtle differences which share similar patterns when objects are moving. Thus, for each image in the training data, we first find its neighbor images with adjacent viewpoints. For each pair of adjacent images, we annotate the association of grasp poses by defining a distance metric. For two grasp poses on the same object, we transform them to the same coordinate frame (*i.e.*, the object frame) and compute two distances:

$$\Delta \mathbf{R} = \arccos \frac{\text{trace}(\mathbf{R}_1^T \mathbf{R}_2) - 1}{2}, \quad (4)$$

$$\Delta \mathbf{t} = \|\mathbf{t}_1 - \mathbf{t}_2\|,$$

where $\mathcal{G}_1 = [\mathbf{R}_1 \ \mathbf{t}_1 \ w_1]$ and $\mathcal{G}_2 = [\mathbf{R}_2 \ \mathbf{t}_2 \ w_2]$ are the transformed grasp poses, $\text{trace}(\cdot)$ is the trace of a matrix, $\Delta \mathbf{R}$ and $\Delta \mathbf{t}$ are the rotation distance and the translation distance respectively. The distance between two grasp poses is defined as

$$d(\mathcal{G}_1, \mathcal{G}_2) = \frac{\Delta \mathbf{t}}{w_{\max}} + \gamma \frac{\Delta \mathbf{R}}{\pi}, \quad (5)$$

where $w_{\max}$ is the maximum gripper width, γ is a distance balancing weight. In practice, we set $w_{\max} = 0.01\text{m}$ and $\gamma = 0.1$. The distance between grasps on different objects is set to infinity.

B. Grasp Perception Model Details

Next, we illustrate the details of our algorithm, which consists of the geometry processing module and the temporal association module. Fig. 2 illustrates the model structure.

1) *Geometry processing module*: Our geometry processing module is built upon previous work GSNet [11], with a minor change made to incorporate the stable score into the network. We first briefly outline the GSNet. Given a point cloud, a 3D convolutional backbone extracts geometric features for each point and also generates a heatmap denoting each point's graspable probability. $\mathcal{M}$ seed points are sampled from the scene according to the graspable probability map ($\mathcal{M}$ is set to 1024). For each sampled point, the network generates 300 view scores to determine which view is more suitable for grasping. The network takes the best view for grasping and

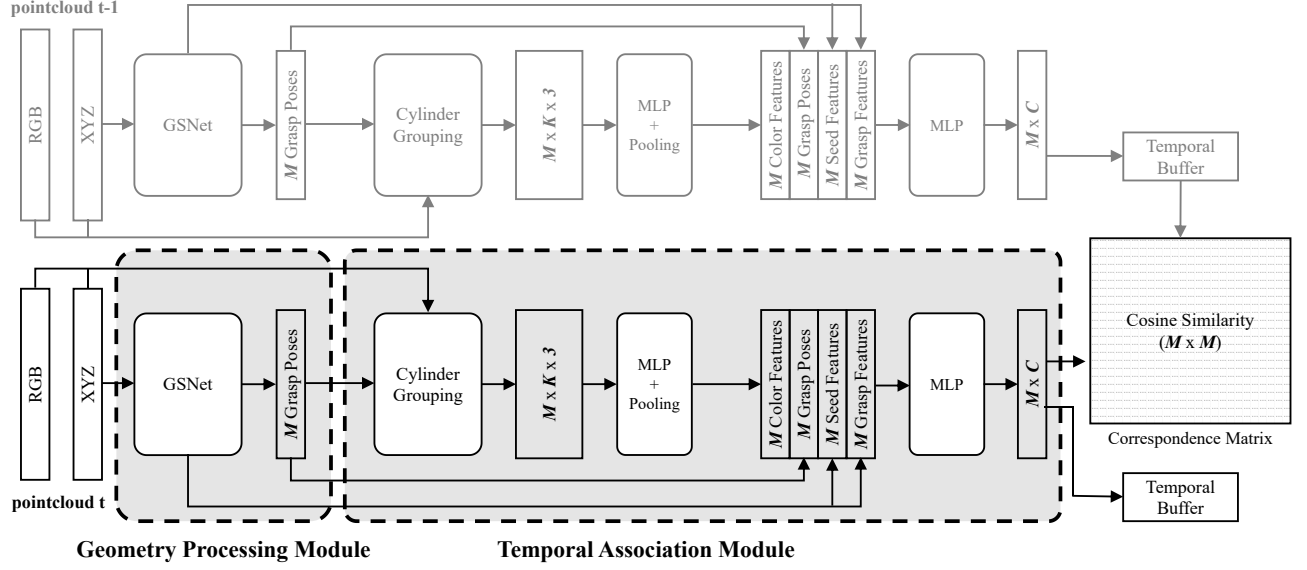


Fig. 2. Illustration of the model architecture for AnyGrasp. For a partial point cloud, the geometry processing module predicts dense grasp poses. The temporal association module generates high-level features for each of the predicted grasp poses. The high-level features are learned with the objective that for the grasp pose pair with smaller distance in the object coordinate frame, the higher cosine similarity they have. Thus we can construct the correspondence matrix for the dense grasp pose pairs across two frames. Details of the model structure are given in the text.

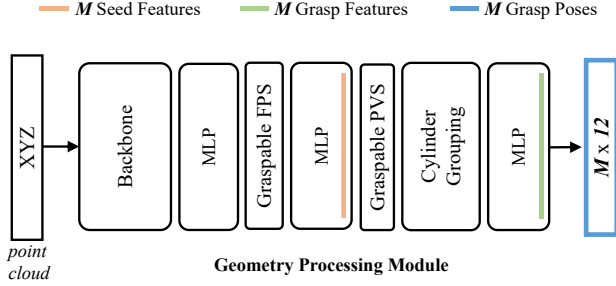


Fig. 3. Detailed illustration of the geometry processing module. The position of the seed features, grasp features and grasp poses are denoted in different colors. The details of each block can be found in [11].

groups the local geometric features along that view within a cylinder space. For each group, the network predicts the grasp scores for 48 grasp poses composed of 12 in-plane rotations and 4 approach depths along that view, as well as 48 grasp widths for these grasp poses. We refer readers to [11] for more details.

The original GSNet predicts $12 \times 4 \times 2$ values for each sampled point at the last layer, representing 12×4 grasp scores and 12×4 grasp widths for the grasp poses along the best view. We modify the last layer and the network now predicts $12 \times 5 \times 2 + 12$ values. The changing from 4 to 5 is because we annotate an extra approach depth for the grasp poses, as stated in Sec. IV-A. The extra 12 predictions denote the stable scores for grasp poses with different angles since the stable score is shared across different grasp depths.

During inference, we multiply the original grasp score by $(1 - \text{stable_score})$ as the new score for a grasp pose.

2) *Temporal association module*: On top of the geometry processing module, we develop a temporal association module to enable grasp pose tracking. The key of the temporal

association module is to generate high-level features for each grasp pose so that we can compute the correspondence score between each two grasp poses across time.

We first introduce how we construct high-level features for each grasp pose. Given an input point cloud, the geometry processing module outputs $\mathcal{M}$ grasp poses by choosing the pose with the highest score on each of the $\mathcal{M}$ seed points. Concurrently, we can obtain the seed features and the grasp features of the seed points. These features are extracted before the last layer of two MLP blocks in the GSNet. Fig. 3 illustrates the feature location. These features mainly represent the geometric cues of the grasp poses. Considering that the texture or color information is also a strong cue for tracking, we also extract the color features of each seed point. Using the predicted grasp pose, we group the RGB information along the grasp direction using cylinder grouping described in [9], where the input seed features are replaced by seed colors. $\mathcal{K}$ RGB tuples are grouped for each of the $\mathcal{M}$ seed points, resulting in a size of $\mathcal{M} \times \mathcal{K} \times 3$. The local texture information is forwarded through an MLP block with pooling layers to obtain the color features. Then the color features, seed features, grasp features, and the grasp poses (parameters of shape $\mathcal{M} \times 12$, the length 12 is composed of rotation matrix (9) and translation (3)) are concatenated and fed into an MLP layer to get the high-level features with size $1 \times \mathcal{C}$ for each predicted grasp.

After obtaining the high-level features of each grasp pose, we can calculate the correspondence score between each two grasp poses. Let $\mathbf{f}_1$ be the feature vector for grasp $\mathcal{G}_1$ in the first point cloud, and $\mathbf{f}_2$ for grasp $\mathcal{G}_2$ in the second point cloud, we calculate the correspondence score using cosine similarity:

$$s_{\text{corres}}(\mathcal{G}_1, \mathcal{G}_2) = \frac{\mathbf{f}_1 \cdot \mathbf{f}_2}{\|\mathbf{f}_1\| \cdot \|\mathbf{f}_2\|}. \quad (6)$$

The scores for all $(\mathbf{f}_1, \mathbf{f}_2)$ pairs are computed and form the

correspondence matrix.

During training, two point clouds from adjacent viewpoints are respectively forwarded through the geometry processing module and the temporal association module to obtain their high-level features for $\mathcal{M}$ grasp poses, both having a size of $\mathcal{M} \times \mathcal{C}$. Then we calculate the correspondence matrix by the cosine similarity. The predicted correspondence matrix with size $\mathcal{M} \times \mathcal{M}$ is compared with the ground-truth matrix and the loss is back-propagated to the temporal association module. The loss function is introduced in the next subsection. During inference, only the point cloud at the current time is required to pass the network, and the high-level features will be stored in a temporal buffer. The correspondence matrix is computed with the current features and those stored in the temporal buffer from the last frame. If we want to track a certain grasp pose selected in the previous step, we can compare its feature vector of size $1 \times \mathcal{C}$ with the $\mathcal{M}$ feature vectors in the current frame and generate an association vector of size $1 \times \mathcal{M}$. Then we pick the feasible grasp with the highest correspondence score as the next prediction.

3) *Loss function*: The loss function for the geometry processing module is introduced in [11]. We refer readers to [11] for more details.

For the temporal association module, two associated grasp poses should have a high corresponding score, which indicates they have similar features. So we adopt supervised contrastive learning [42] in model training, which pulls together the features from the same class.

For two grasp poses $\mathcal{G}_1$ and $\mathcal{G}_2$, they are treated as the same class if and only if $d(\mathcal{G}_1, \mathcal{G}_2) \leq \sigma$. The predicted grasp sets for two point clouds are denoted by $\mathbf{G}_1 = \{\mathcal{G}_i^1 | i = 1, 2, \dots, M\}$ and $\mathbf{G}_2 = \{\mathcal{G}_j^2 | j = 1, 2, \dots, M\}$ respectively. For a grasp pose $\mathcal{G}_i^1$ in the first point cloud, we collect all the grasp poses in the second point cloud belonging to the same class of $\mathcal{G}_i^1$, which are denoted by $\mathbf{P}(i) = \{\mathcal{G}_k^2 \in \mathbf{G}_2 | d(\mathcal{G}_i^1, \mathcal{G}_k^2) \leq \sigma\}$. The loss function is defined as

$$L = \sum_{\mathcal{G}_i^1 \in \mathbf{G}_1} \frac{-1}{|\mathbf{P}(i)|} \sum_{\mathcal{G}_k^2 \in \mathbf{P}(i)} \log \frac{\exp(s_{\text{corres}}(\mathcal{G}_i^1, \mathcal{G}_k^2)/\tau)}{\sum_{\mathcal{G}_j^2 \in \mathbf{G}_2} \exp(s_{\text{corres}}(\mathcal{G}_i^1, \mathcal{G}_j^2)/\tau)}, \quad (7)$$

where $|\mathbf{P}(i)|$ stands for the cardinality of $\mathbf{P}(i)$ and τ stands for the temperature parameter.

C. Training Details

Input point clouds are down-sampled with a voxel size of 0.005m. In the temporal association module, we set $\mathcal{M} = 1024$ for each point cloud. The two modules are trained on the extended GraspNet-1Billion dataset using one Nvidia GTX 2080 Ti GPU with Adam optimizer [43] and an initial learning rate of 0.001. We adopt the “poly” policy with *power* = 0.9 for learning rate decay, which is used in DeepLab [44].

We first train the geometry processing module from scratch with a batch size of 4. For data augmentation, we randomly flip the scene horizontally, randomly rotate the points by Uniform $[-30^\circ, 30^\circ]$ around the Z-axis (in the camera co-ordinate frame). We also randomly translate the points by Uniform $[-0.2, 0.2]$ m in X/Y-axis and Uniform $[-0.1, 0.2]$ m in Z-axis.

After the geometry processing module converged, we freeze its weights and train the temporal association module jointly with it. Each mini-batch contains four pairs of point clouds, where the two point clouds in one pair are captured by neighboring viewpoints from the same scene. Besides random flipping, random rotation and random translation, we also randomly remove some objects in the scene for data augmentation. In the loss function, we set $\sigma = 0.1$ and $\tau = 0.1$.

D. Detection Post-processing

For the detected grasp poses, we conduct two post-processing to improve the stability. The first step is to perform collision detection. Although our network also implicitly learns whether a grasp pose would collide with the scene, such obstacle-aware property is not a hard constraint and may be prone to noise. Thus, we perform extra collision detection for the top-100 grasp poses among the predicted results. The collision detection is based on the partial-view point cloud by examining whether there is any point within the gripper-occupied grid. This step provides a safety guarantee in most cases.

The second step is to perform a gripper-centering process. It comes from an observation that if the two fingertips of the gripper contact the object surface sequentially, the earlier contact may push the object away and cause a failed grasp. The main reason is that the GraspNet-1Billion dataset does not restrict the gripper fingertips to have the same distance to the object. Thus, we perform the gripper centering process by calculating the distance from both fingertips to their contact points and translating the gripper along the connection direction of the fingertips to ensure the same moving distance from both fingertips to their contact points.

The above two processes are implemented on GPU with matrix computation and take 80 ms for 100 grasp poses.

V. EXPERIMENTAL SETUP

To verify the performance of our grasp perception system, we embed it with real robot platforms and conduct grasping experiments.

A. Hardware and Human Subject

For the static scene grasping, we use a UR5 robot arm with an overhead camera. Intel RealSense D415 and D435 are adopted to evaluate the algorithm performance across depth sensors. We use a Robotiq-85 gripper. In [7], the authors used a customized silicone soft fingertip [45] that is designed for robust grasping. Since it is not available to us, we attach a soft table tennis rubber¹ to the fingertips, which is publicly available to any research group.

We also invite human subjects to conduct the bin picking and compare it with the AnyGrasp perception system. Human subjects are required to use a two-finger jaw for a fair comparison. The two-finger jaw has the same opening width as the robot gripper. Note that since we focus on the visual

¹https://www.amazon.com/Rubbers-DHS-Table-Tennis/s?rh=n%3A3419421%2Cp_89%3ADHS

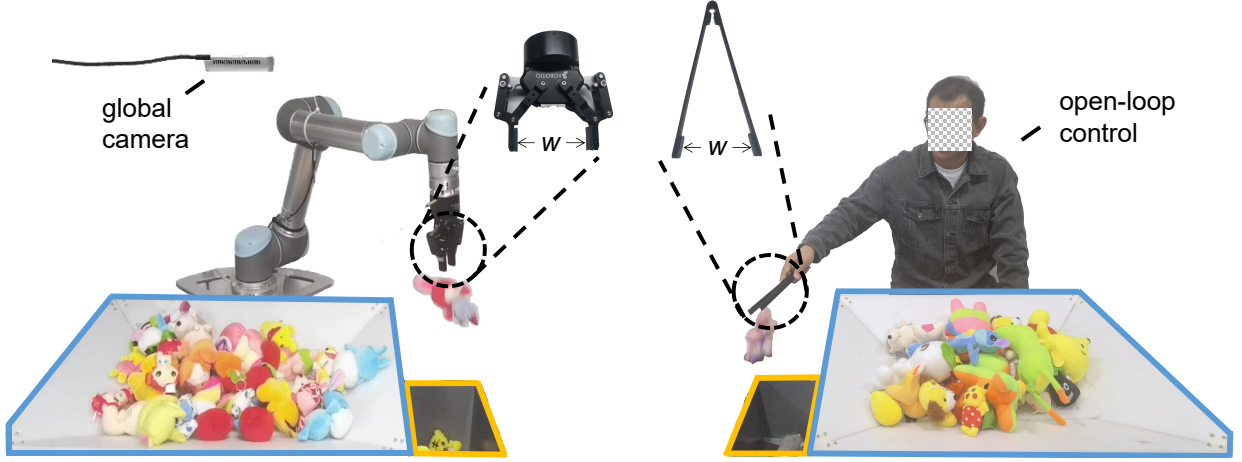


Fig. 4. Illustration of the hardware and volunteer setting in the static scene. Objects are moved from the object plate (surrounded by a blue line) to the target bin (surrounded by a yellow line). The robot is equipped with a global camera. We ensure the same fingertip material and the same opening width of the robot gripper and the human-held jaw. The human subjects are asked to perform an open-loop control, where adjusting the grasp pose with tactile feedback after contacting the object is not allowed.

perception for grasping, we ask the volunteers to adopt an open-loop strategy during grasping. It means that they shall not adjust the grasp pose with tactile feedback after contacting the object. To enable fair comparison with human grasping, we attach the same soft rubber to the human hold jaw. Fig. 4 shows the robot and human subject settings.

For dynamic grasping, we use a Flexiv Rizon arm since it can update the servo targets more smoothly. An Intel RealSense L515 camera is attached to the wrist of the robot. We adopt the in-hand setting since the overhead camera might be occluded by the robot during tracking and choose the L515 since it has a larger depth range and can work robustly when the object is close to the camera. The D415 or D435 depth camera cannot generate depth information when the object distance to the camera is less than 15 cm. For safety reasons, we extend the fingertips of the Robotiq-85 gripper by mounting two extra 3D-printed parallel jaws. Fig. 5 illustrates the hardware setting in this experiment.

All the models in the following experiments run on a workstation with Ubuntu 20.04 system, Intel i9-10900K CPU and Nvidia 2060 GPU. The codes are written in Python.

B. Experimental Procedure

For the parallel-jaw bin picking experiments, we conduct two independent trials for each experiment. The camera extrinsic parameters to the robot are obtained by Aruco marker detection and extra human measurement. In each trial, we randomly pour the objects onto the plate without too much human intervention. For each grasp attempt, our algorithm receives a single-view point cloud and predicts abundant grasp poses for the scene. For safety, we set a workspace limit. The limit includes the restriction on the plate plane (area enclosed by the blue dashed line on the plate, see Fig. 9) and the restriction on grasp pose orientation. Specifically, we restrict the angle between the grasp approach direction and the vertical direction to be less than 25° , as we empirically found that a larger angle may result in an unreachable pose for the

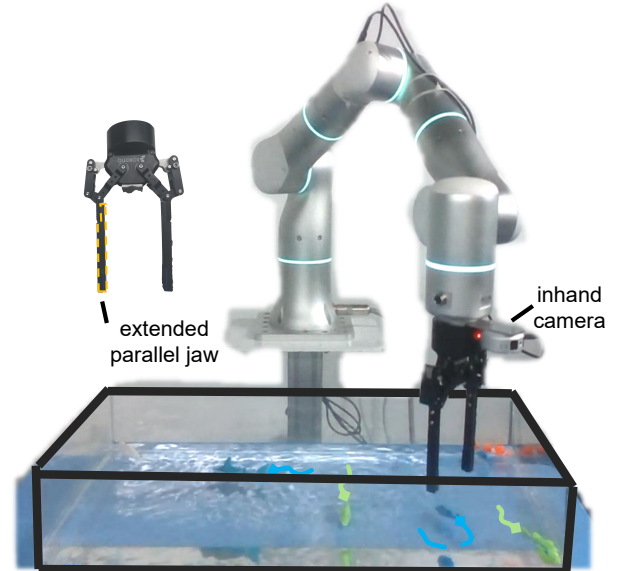


Fig. 5. Illustration of the hardware setting in the fish-catching task. An in-hand camera is attached to the robot's wrist. The moving trajectories of the robot fish are visualized with colored lines. We attach two 3D-printed parallel jaws to the Robotiq gripper.

UR5 arm. The grasp pose without occlusion, located within the workspace and with the highest grasp score is selected.

To obtain a trajectory without collision with the scene during grasping, we set a waypoint pose by translating the grasp pose 10 cm backward along its approach direction. The waypoint and grasp poses are sent to the UR5 robot arm through UR script command `moveLs()` via Ethernet and the motion planner of UR5 would move the gripper to these poses sequentially. We found that this simple strategy can avoid most collision situations. The gripper will close after the robot reaches the target pose, grasp the object, and then move it to the target bin. During the grasping procedure, the objects may scatter outside of the plane workspace. We would manually push these objects back. The operator will record whether the

Algorithm 1 Dynamic Grasping Algorithm

```

1: Initialize: robot.move( $\mathbf{T}_{\text{ready}}$ ), open gripper, empty buffer
2: Mark  $\mathbf{G}_0$  as empty
3: for time step  $j \leftarrow 1$  to  $\infty$  do
4:    $\mathbf{T}_j^{\text{tcp}} \leftarrow \text{robot.pose}$ 
5:    $\mathcal{P}_j \leftarrow \text{camera.perception}$ 
6:    $\mathbf{G}_j \leftarrow \text{GetGraspPose}(\mathcal{P}_j)$ 
7:   if  $\mathbf{G}_j$  is empty then ▷ lost tracking of objects
8:     robot.servo( $\mathbf{T}_{\text{ready}}$ ), empty buffer
9:     continue
10:  end if
11:  if  $\mathbf{G}_{j-1}$  is empty then ▷ for the first frame
12:     $\mathcal{G}_j^* \leftarrow \arg \max_{\mathcal{G}_j \in \mathbf{G}_j} \text{Prob}(s = 1 \mid \mathcal{P}_j, \mathcal{G}_j)$ 
13:  else ▷ for the following tracking frames
14:     $\mathcal{G}_j^* \leftarrow \arg \max_{\mathcal{G}_j \in \mathbf{G}_j} s_{\text{corres}}(\mathcal{G}_{j-1}^*, \mathcal{G}_j)$ 
15:  end if
16:  Add  $\mathcal{G}_j^*$  into buffer and predict  $\tilde{\mathcal{G}}_j^*$  from buffer
17:  Calculate  $\Delta R, \Delta t$  and  $\Delta t_{\text{xOy}}$  between  $\tilde{\mathcal{G}}_j^*$  and  $\mathbf{T}_j^{\text{tcp}}$ .
18:  if  $\Delta R \leq \delta_R$  and  $\Delta t \leq \delta_t$  and  $\Delta t_{\text{xOy}} \leq \delta_{t_{\text{xOy}}}$  then
19:    ▷ grasp when gripper is close enough to the fish
20:    robot.move( $\tilde{\mathcal{G}}_j^*$ ), gripper.close()
21:    Move robot to the throw pose and open gripper
22:    Re-initialize the system. ▷ See L1 for details
23:  else ▷ continue tracking with pregrasp pose
24:    Calculate pregrasp pose  $\mathbf{T}_j^p$  using  $\tilde{\mathcal{G}}_j^*$  and  $d_{\text{pregrasp}}$ 
25:    robot.servo( $\mathbf{T}_j^p$ )
26:  end if
27: end for

```

object is successfully moved to the target bin. Each trial of experiments is regarded as finished if the algorithm cannot estimate grasp poses for the scene ten times.

For the human bin-picking task, two volunteers are invited to conduct the grasping experiments and each is required to clear the plate twice.

For the dynamic fish-catching experiment, we conduct five independent trials. The camera extrinsic is obtained by measuring the customized camera holder. The pseudo-code of our dynamic grasping algorithm is illustrated in Alg. 1. In each fish-catching trial, we first generate and select a grasp pose $\mathcal{G}_1^*$ for the first frame using the same procedure in the static scene. For simplicity, we regard the grasp pose has been transformed to the robot coordinate system using the camera extrinsic parameters. Then the robot enters the pre-grasp servoing stage. The grasp perception system keeps active during this process and generates updated grasp pose $\mathcal{G}_j^*$ of time step j that has a high association score with the previous grasp pose $\mathcal{G}_{j-1}^*$. Empirically, we found that it is important to predict the future grasp pose when catching the moving fish, since it takes some time to close the gripper. Hence, the selected grasp poses during the tracking procedure are saved to a temporary buffer of length 10 for future grasp pose prediction. The system will predict a future grasp pose $\tilde{\mathcal{G}}_j^*$ by adding the moving momentum calculated from the temporary buffer to the last grasp pose $\mathcal{G}_j^*$. In the pre-grasp servoing stage, we do not directly servo to the future grasp pose. Instead, the servoing target pose $\mathbf{T}_j^p$ during the pre-

grasp stage is obtained by translating $\tilde{\mathcal{G}}_j^*$ along the z -axis by $d_{\text{pregrasp}} = 3.5\text{cm}$ backward. This is to avoid touching the moving fish and changing their motion status during servoing. Meanwhile, we only preserve the rotation along the z -axis of the gripper and eliminate the rotation along the other two axes to improve the tracking stability. The servoing target $\mathbf{T}_j^p$ is sent to the Flexiv arm through Flexiv RDK and the robot would servo to the target pose. The ending criterion for the servoing process is that the 3D distance Δt between the predicted future grasp pose $\tilde{\mathcal{G}}_j^*$ and the robot's current end-effector pose is less than $\delta_t = 5.5\text{cm}$, the 2D distance Δt_{xOy} in the horizontal plane is less than $\delta_{t_{\text{xOy}}} = 2\text{cm}$, and the angle ΔR between two poses is less than $\delta_R = 20^\circ$. The robot will then enter the grasping stage which moves to the grasp pose and closes its gripper. If the tracked pose moves outside of the robot's workspace or camera's view, the robot will return to the initial state, empty the temporary buffer and choose a new grasp target.

VI. EXPERIMENTAL RESULTS

In this section, several experiments are demonstrated to evaluate the following properties of our full-DoF grasp perception system: (i) generalization ability to different objects and sensors; (ii) accuracy on different kinds of objects compared with humans; (iii) temporal consistency for random moving objects; (iv) efficiency of the whole perception system.

A. Static Scenes

We first conduct grasping experiments in static scenes. To build a representative test set that can fully evaluate the grasp perception system, we collect several common categories of daily-life objects. In total, over 300 objects are collected. Fig. 6 (a) gives an overview of the objects. The size of the objects ranges from $1.5 \times 1.5 \times 1.5 \text{ cm}^3$ to $36 \times 4 \times 11.5 \text{ cm}^3$.

We first compare the grasping system on daily objects with human operators using the same end-effector configuration and a previous method [7]. The overall grasping results on different objects are presented in Fig. 6 (b). The videos can be found in the Appendix B (Movie S1-S4). From Fig. 6 (b), we can see that our perception system is robust towards different categories of objects. On the large scale benchmark, its accuracy ranges from 83.3% to 100% on different kinds of objects, yielding 93.3% on average. Such grasping accuracy is on-par with human subjects, which is 93.9% on average. However, human subjects give more stable performance on different kinds of objects. The grasp accuracy ranges from 91.6% to 96.6%.

For each trial, the grasp perception system predicts the grasp poses in 100ms and the overall grasp decision time (including collision detection and pose adjustment) is less than 200 ms. Thus, the mean pick per hour (MPPH) only depends on the robot moving and gripper executing speed. With a single UR5 arm that has a maximum end-link speed of 1m/s and a Robotiq gripper, our grasping system achieves over 900 MPPH, which is a dramatic improvement compared to previous state-of-the-art [7], 300 MPPH achieved by a dual-arm system. Such a metric can be further optimized in the industry using a higher-speed robot arm and gripper. We also observe that humans can

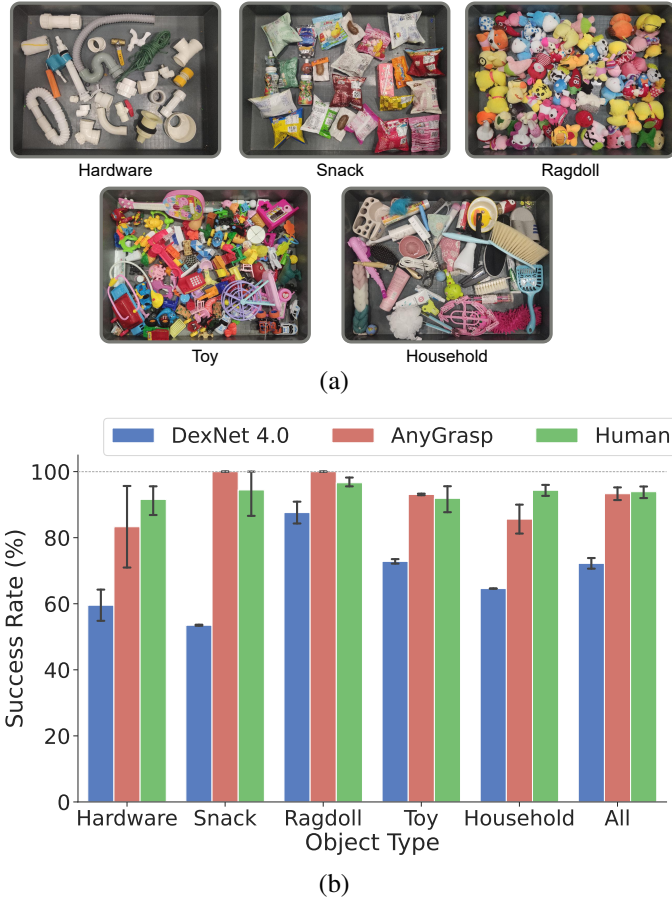


Fig. 6. (a) Example objects from the unseen object test set. (b) The grasp success rate of our system compared to a previous grasp pose detection system and the human's performance using a parallel jaw.

achieve over 1,500 MPPH with their maximum speed, and on average they can achieve 1,000-1,200 MPPH.

We then show the performance of our algorithm on different depth sensors. We visualize the typical error maps of the two cameras we use in Fig. 7 (a). The error maps are obtained by subtracting the distance between the camera and the captured flat plane from the measured depth maps. We can see that the depth given by the D435 camera presents a larger variance and can achieve up to 15mm error. Even with this noise, our algorithm can still perform well, as illustrated in Fig. 7 (b). The video of grasping using a D435 camera can be found in the Appendix B (Movie S5). The experiments demonstrate the robustness of our algorithm towards depth sensing noise, mainly owing to the training data collected with real sensors.

We further evaluate our grasp perception system on a challenging adversarial object set, which contains the 13 adversarial objects used in DexNet2.0 [5] and the 49 evaluation objects from EGAD [46]. The objects from the EGAD evaluation set are generated by the program to ensure the shape complexity and grasp difficulty. Fig. 8 shows the examples of the adversarial object set and the statistics of the success rate. Full videos can be found in the Appendix B (Movie S6-S8, part of S3-S4). We can see that the accuracy decreases for our algorithm and the previous method [7]. However, humans

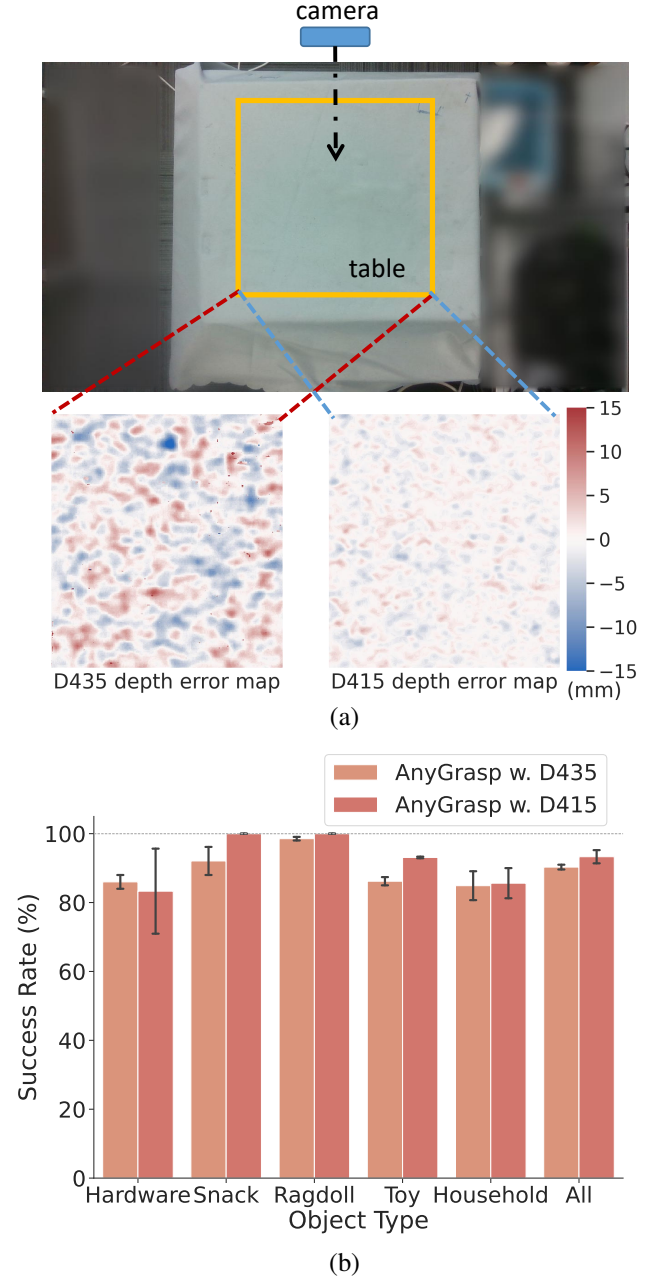


Fig. 7. (a) The depth error map of two frequently adopted depth cameras. The distance unit is millimeter. (b) The grasp success rate comparison of AnyGrasp when using these two cameras.

can still perform stably. The main reason is that our grasp perception system would repeat the failed trials on an object without utilizing the feedback from each trial. It deserves more exploration in the future.

Finally, we set a challenging real-world task, where the robot is required to clean the fragments of a broken clay pot. It is challenging since the fragments are pretty thin, usually less than 3mm, which requires an accurate estimation of the grasp pose. The noisy depth perception further increases the difficulty. We haven't seen previous grasp pose detection algorithms demonstrate success in this scenario. Fig. 9 shows the snapshots of a robot cleaning the thin fragments under the guidance of the AnyGrasp perception system and the video is

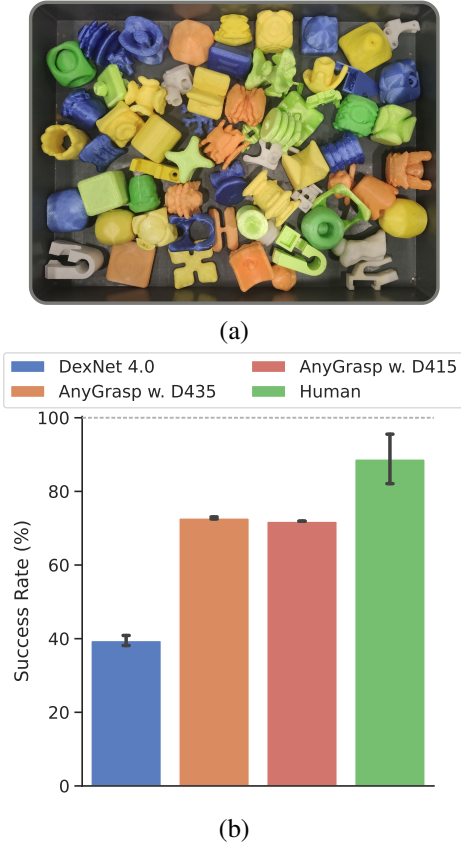


Fig. 8. (a) The 3D-printed adversarial objects. It can be seen that the object’s surface is quite smooth. (b) The grasp success rate of different methods on the challenging adversarial object set.

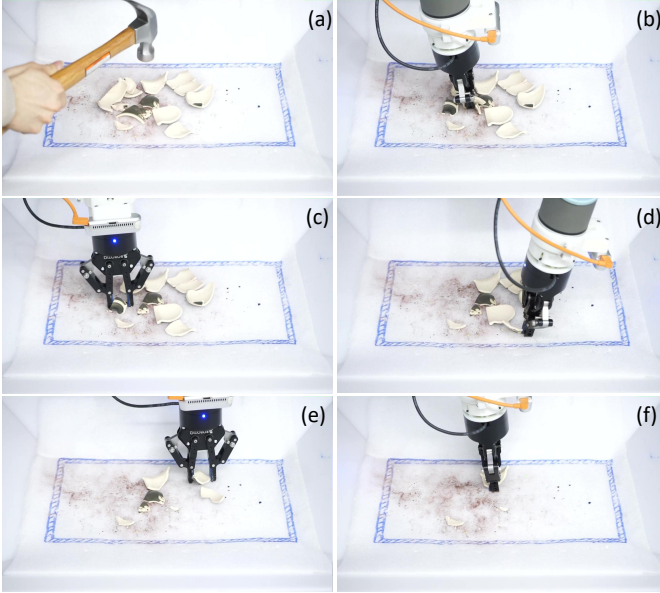


Fig. 9. Snapshots of the fragments cleaning task. It requires the grasp pose detection model to generate accurate poses for the thin pieces. The sequence order is from (a) to (f).

given in Appendix B (Movie S9).

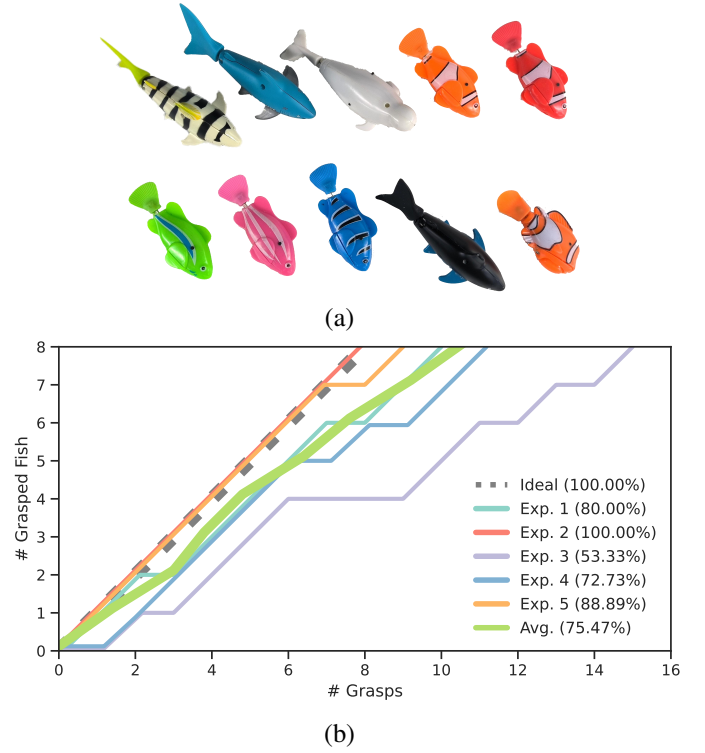


Fig. 10. (a) Different robot fish were used in our experiments. (b) Performance of the fish catching experiments.

B. Dynamic Scenes

To verify the temporal consistency of the grasp perception system, we conduct a robot fish-catching experiment. A frequently used heuristic baseline [23] that keeps tracking the nearest grasp poses across frames is also implemented and compared with our temporal association module.

Previous literature examines the grasp pose tracking method in a human-robot hand-over setting [33]. For our fish-catching experiment, it poses extra challenges for the tracking system: (i) the underwater environment greatly decreases the contact friction between the gripper and the robot fish; (ii) the fish is pretty small and tolerates small final pose errors, while the handing object is usually larger; (iii) humans tend to stabilize the objects when the robot is about to grasp, while the fish keeps moving in the whole grasping process; (iv) the point cloud would be noisier due to the light reflection and refraction in water.

In our experiment, we randomly place 8 robot fish in the fish tank each time. Fig. 10 (a) shows different robot fish used in our experiments. Note that we have multiple instances for each kind of fish. The robot will try to catch all the swimming fish, and we record the success rate during grasping. The whole procedure is repeated 5 times, and we show the detail performance in Fig. 10 (b). We illustrate the process of the robot catching a fish in Fig. 11 and the full video can be found in Appendix B (Movie S10). The average success rate achieves 75.5%. We can see that our grasp perception system is robust in this challenging dynamic grasping scenario.

We further conduct a failure analysis and found that the failed cases can be divided into five categories, as shown in

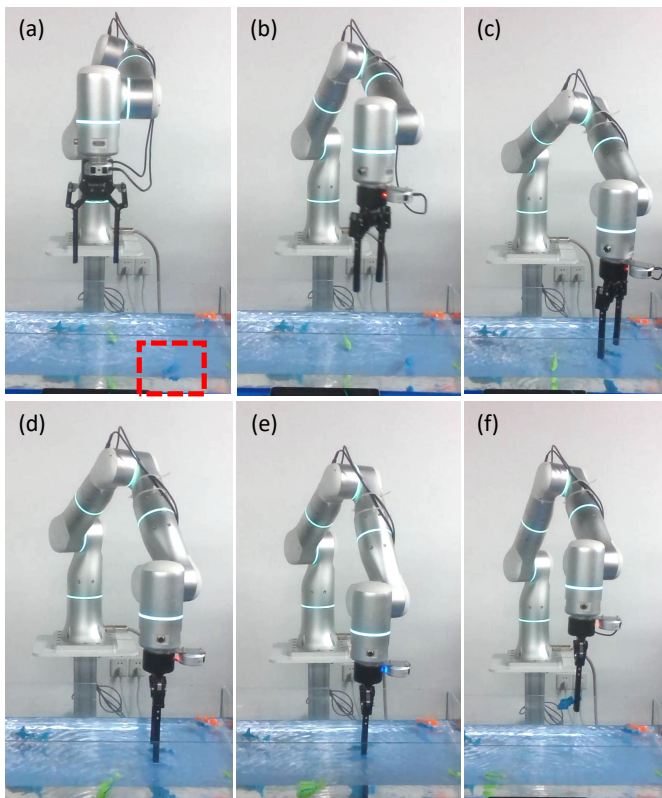


Fig. 11. Snapshots of the robot fish-catching procedure: (a) robot selects a grasp target, denoted by a red dashed line; (b)-(e) robot serves to the target grasp pose while updating the pose; (e) robot closes the gripper; (f) robot lifts the grasped fish.

Fig. 12. In nearly a half of the failure cases, the fish slips away although the grasp pose is good, mainly due to the small friction in the water. The second and fourth reasons are that the predicted future grasp pose falls in front of or behind the fish. The main reason is that the robot fish would change its speed during moving and the historical momentum is outdated. The third reason is that the grasp quality is not good enough, and the gripper finger would push the fish away during closing. The last reason is the correspondence switch, which happens when there are two close and similar fish. The correspondence switch would inject noise into the history pose buffer and lead to a wrong predicted future grasp pose.

On the other hand, the heuristic method achieves an average success rate of 62.5%. During grasping, we found that this nearest target policy is more possible to fall behind the moving target during tracking, and takes a longer time to enter the grasping state. Specifically, the baseline method takes 12.7% more time than our method on the successful grasps on average.

Overall, the grasp perception system can run at 7 Hz on an Nvidia 2060 GPU.

VII. DISCUSSION

A. Train in Simulation

As most methods and datasets provide training data in simulation, we try to figure out the performance gap between training on real data and simulated data. Since our dataset

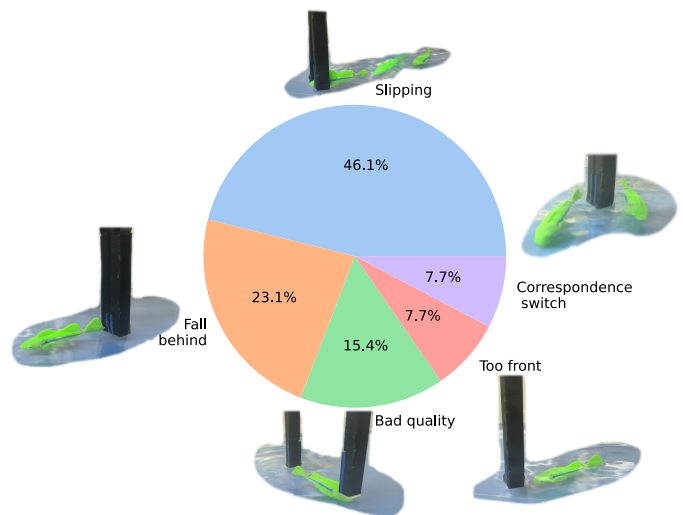


Fig. 12. Failure analysis in the fish catching experiments. We illustrate each failure case with an example image. Zoom in for more details and see the text for more explanation.

contains object 3D models and their 6D poses in the scene, we obtain simulated RGBD images by rendering the scene in PyRender [47]. The same model is trained on these simulated depth images. We also try the frequently adopted data augmentation technique that adds Gaussian noise to the depth images [5], [7] for better sim-to-real transfer.

We first analyze the detailed performance on the GraspNet-1Billion benchmark, which directly evaluates the grasp pose's quality based on object mesh using the force-closure analysis. The original metric computes the averaged AP for the Top-50 predicted grasp poses in the scene. To show more details, we compute the averaged AP for grasp poses ranging from Top-1 to Top-50. The performance on different test splits for our network trained with different strategies is shown in Fig. 13. We can see that adding Gaussian noise to the point cloud can improve the performance during evaluating, but still remains a large performance gap with directly trained on real data. Besides, the performance gap between training on real data and simulated data also increases when the test set contains harder objects (the novel test scenes), especially for the high-score grasp poses.

Then we evaluate the model trained on simulated data with Gaussian noise in the real-world bin-picking experiment. The results are shown in Fig. 14. Movie S11 in the Appendix B records the grasping process. We can see that the performance decreases a lot compared to the original model. Besides the lower success rate, we also observe that the network cannot generate grasp poses for a scene when there remain few objects. We conjecture the model trained in the simulation would generate fewer grasp poses with high scores in the real world due to the domain shift.

B. Dense Supervision Strategy

Besides providing data captured in real-world, another difference between GraspNet-1Billion [9] and other datasets [5], [46], [48], [49] is that the grasp pose annotation is much

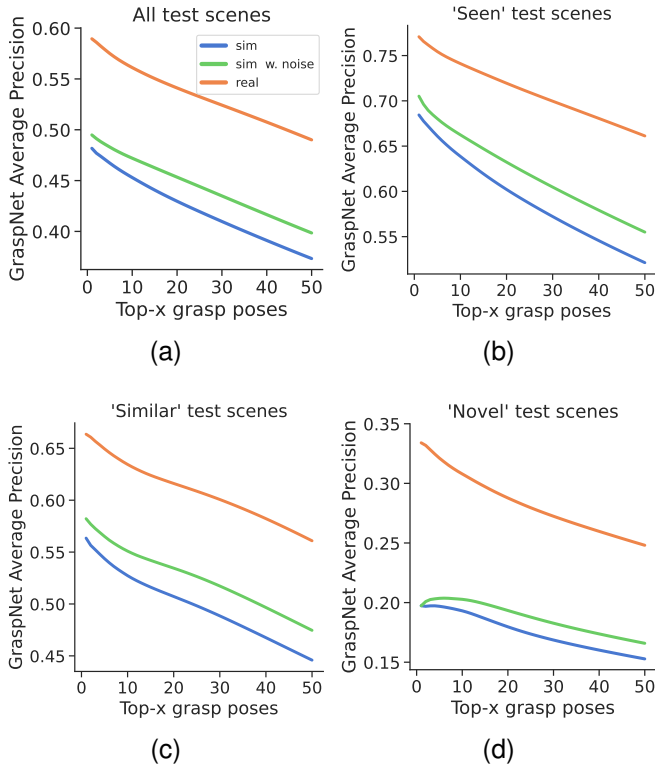


Fig. 13. Evaluation results on the GraspNet-1Billion test set when training with simulated and real-world data. From (a) to (d) we present the average precision (AP) on the whole test set, the test set with seen objects in the training set, the test set with similar objects in the training set, and the test set with novel objects. “sim” denotes the model trained with perfect simulated depth images, “sim w. noise” denotes the model trained with simulated depth images with Gaussian noise.

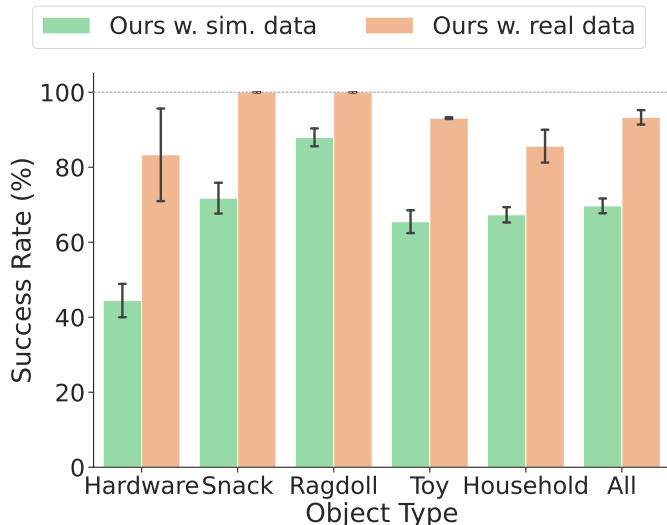


Fig. 14. The success rate of our system when training with real and simulated data. Note that here the “sim. data” denotes the simulated depth images with extra Gaussian noise.

denser in [9]. For examples, Jacquard [48], DexNet 2.0 [5] and EGAD [46] generates 100 grasp poses on each object, ACRONYM [49] generates 2k grasp poses and [8] generates 34k grasp poses. In contrast, the GraspNet-1Billion generates

over 10M grasp poses on each object. Here we also evaluate the influence of this factor. We train multiple models on the dataset with different numbers of annotations. The models are evaluated on the GraspNet-1Billion test set with the same metric introduced in the above subsection.

In Fig. 15, we show the model evaluation results with different training configurations. We downsample the training set at different dimensions, namely the grasp pose density on each object, the image amount, and the scene amount. These dimensions are uniformly downsampled by 10 times and 50 times respectively. We can see that when downsampling the grasp poses on objects by 10 times, the performance degradation is similar to decreasing the number of training images. Meanwhile, the performance decreases more on the novel object test scenes. When we downsample the training samples by 50 times, we can see that the degradation is still similar for the grasp pose dimension and the image dimension. These results suggest that the densely annotated grasp poses are as equal importance as the number of training images. Considering that they can be annotated automatically without extra human efforts in collecting images, it is a good trade to annotate dense grasp poses on objects.

Another interesting finding is that when we downsample the scene dimension of the training data, the model performance shows larger degradation on the test sets. When we downsample the scene by 50 times (namely 2 training scenes are used), the model cannot converge. It suggests that the diverse scene might be more important for model training. We hope our analyses would be beneficial to future dataset design.

C. Comparison with 6D Pose Tracking

Recently, there is also research focusing on novel object 6D pose tracking [50]. Thus, it may be intuitive to adopt an unseen object 6D pose tracking pipeline for grasp tracking. Here, we explain the advantages of grasp pose tracking versus object 6D pose tracking. Firstly, the 6D pose tracking algorithm cannot handle deformable objects, and thus is not applicable for grasping in such cases. Secondly, the 6D pose tracking algorithm may be prone to large depth noise like incomplete point cloud, since it focuses on the object level features. But the grasp tracking algorithm can work robustly for a target grasp pose, even though other parts of the object are heavily occluded. In Fig. 16 we show an example illustrating these two cases.

D. Closed-loop Grasp Adjustment under Occlusion

Humans conduct closed-loop grasping in daily manipulation. Although our algorithm is also temporal continuous and can enable closed-loop grasping, it is fragile to the occlusion induced by the robot. When the robot approaches the object, the gripper would occlude the grasp target. Thus, the visual perception would fail. How to enable the closed-loop grasp perception against occlusion is an open question. A missing piece here is tactile perception. It aids grasp slip detection and enables in-hand adjustment. This ability is out of the scope of this paper and will be our future work.

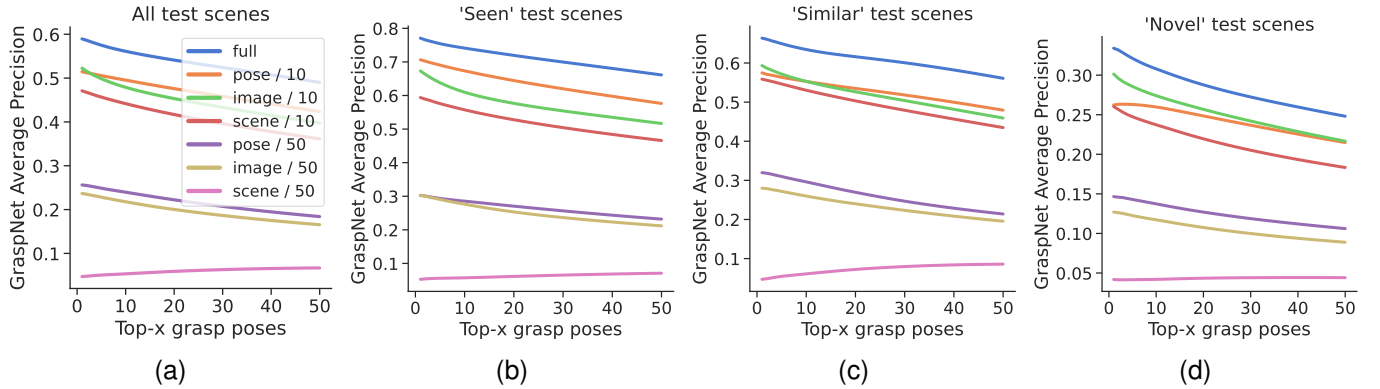


Fig. 15. Evaluation results on the GraspNet-1Billion test set when training with different portions of real-world data. “pose / 10” denotes we downsample the grasp pose density on each object by 1/10. “image / 10” denotes that we downsample the training images by 1/10. “scene / 10” denotes that we downsample the training scene by 1/10. So are the meaning of “pose / 50”, “image / 50” and “scene / 50”.

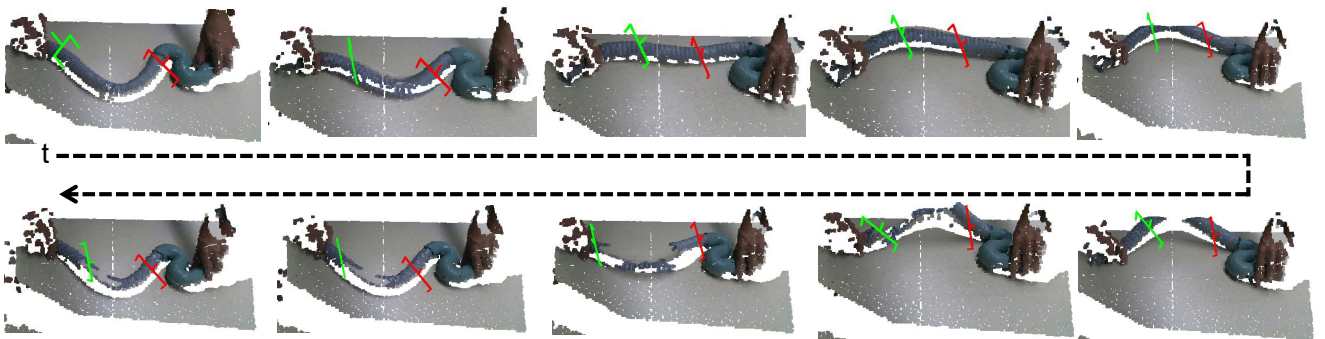


Fig. 16. Illustration of tracking two sampled grasp poses on a deformable, textureless tube. We can see that the grasp poses are stable in this case, even under severe noise input. The object 6D pose tracking algorithm cannot handle this scenario.

VIII. CONCLUSION

In this paper, we presented AnyGrasp, a visual grasp perception system that can generate spatially dense and temporally smooth grasp poses. We proposed a unified model, wherein the dense prediction of the geometry processing module is the basis to enable smooth grasp pose tracking in the temporal association module. The collisions and object COG were implicitly learned through the supervision signal. Owing to our learning strategy with real-world data, the model performs robustly against different real-world depth sensor noise. Various experiments were conducted to verify the accuracy, robustness, and efficiency of our method. Finally, we analyzed the influence of several factors in the dataset design.

Although our method provides the closed-loop grasping ability, it cannot adjust the grasp pose using visual or tactile feedback. This ability can help the robot to recover from previous errors soon. Meanwhile, we only focused on visual grasping for the two-finger parallel gripper in this section. It would be significant to transfer such ability to different robotic hands. Our future research will be conducted accordingly.

ACKNOWLEDGMENTS

We would like to thank Jiang Zou, Xiaolin Fang for insightful discussions and comments, and Qiaojun Yu for helpful feedback on this work.

Authors' contributions: H.-S.F. designed the experiments, devised the training method, implemented the static grasping experiments, collected the data and wrote the manuscript. C.W. implemented the neural network training, annotated the grasp label and edited the manuscript. H.F. implemented and conducted the dynamic grasping experiments. M.G. helped the data collection, wrote the API for data parsing and built the robot platform. J.L. generated the simulated data and assisted real-robot experiments. H.Y and W.L assisted real-robot experiments. Y.X. assisted the network training. C.L. supervised the project and provided the environment and funding support for this research.

APPENDIX

A. Object Collection

To comprehensively evaluate the algorithm performance in daily scenario, we decide to collect a fairly large scale test object set, as we found that the results varied on different objects. To construct the object set, we go to the supermarket, the hardware store and the toy store for procurement. The principle for selecting objects is that there exists some graspable place smaller than the gripper width on a object. In total, over 300 objects are collected, which is an order of magnitude larger than previous practice [7]. We believe such a large object test can provide a thorough evaluation to algorithms. Note that

we do not collect objects with a large portion of transparent or black surface, since current depth sensor cannot give good prediction on these materials. Some recent papers [51], [52] focused on solving this problem with RGB information, but it is not fully addressed yet. The challenging adversarial object set is 3D printed with poly-lactic acid material. This results a smooth surface and makes the grasping even harder.

B. Supplementary Videos

We record all the real robot experiments conducted in this paper. All the videos are uploaded to YouTube and will be permanently stored in support of this paper. The original length of the videos is over 12 hours. They are speeded up according to their importance to the paper. We honestly report the statistics of each video. Below we provide the links to these supplementary videos.

- S1: “AnyGrasp Demo: RealSense D415 camera on daily objects”, <https://youtu.be/dNnLgAGrec>
- S2: “DexNet4.0 tested on daily objects”, https://youtu.be/vDqsrj_rtk8
- S3: “AnyGrasp experiment: Human subject 1 on daily objects and adversarial objects”, <https://youtu.be/-h0yvFZDfko>
- S4: “AnyGrasp experiment: Human subject 2 on daily objects and adversarial objects”, <https://youtu.be/yItVN-Awjrg>
- S5: “AnyGrasp Demo: RealSense D435 camera on daily objects”, <https://youtu.be/7pgdbyLN0A4>
- S6: “DexNet4.0 tested on adversarial objects”, <https://youtu.be/9vOop28YReg>
- S7: “AnyGrasp Demo: RealSense D435 camera on adversarial objects”, https://youtu.be/AK_nHgH4RBA
- S8: “AnyGrasp Demo: RealSense D415 camera on adversarial objects”, <https://youtu.be/8FztVFRcvMY>
- S9: “AnyGrasp Demo: Cleaning fragments of a broken pot”, <https://youtu.be/s0SUwlvgr8>
- S10: “AnyGrasp Demo: Robot Fish Catching by A Robot”, <https://youtu.be/2O7UoOxeLlk>
- S11: “AnyGrasp trained solely in simulation, tested in real world”, <https://youtu.be/IBKoa9OAI0A>

REFERENCES

- [1] D. Baldauf and H. Deubel, “Attentional landscapes in reaching and grasping,” *Vision research*, vol. 50, no. 11, pp. 999–1013, 2010.
- [2] A. Bicchi and V. Kumar, “Robotic grasping and contact: A review,” in *IEEE International Conference on Robotics and Automation (ICRA)*, vol. 1. IEEE, 2000, pp. 348–353.
- [3] J. Bohg, A. Morales, T. Asfour, and D. Kragic, “Data-driven grasp synthesis—a survey,” *IEEE Transactions on Robotics (T-RO)*, vol. 30, no. 2, pp. 289–309, 2014.
- [4] J. Redmon and A. Angelova, “Real-time grasp detection using convolutional neural networks,” *Proceedings IEEE International Conference on Robotics and Automation (ICRA)*, 2014.
- [5] J. Mahler, J. Liang, S. Niyaz, M. Laskey, R. Doan, X. Liu, J. Aparicio, and K. Goldberg, “Dex-net 2.0: Deep learning to plan robust grasps with synthetic point clouds and analytic grasp metrics,” in *Proceedings of Robotics: Science and Systems (RSS)*, Cambridge, Massachusetts, July 2017.
- [6] A. ten Pas, M. Gualtieri, K. Saenko, and R. Platt, “Grasp pose detection in point clouds,” *The International Journal of Robotics Research (IJRR)*, vol. 36, no. 13-14, pp. 1455–1473, 2017.
- [7] J. Mahler, M. Matl, V. Satish, M. Danielczuk, B. DeRose, S. McKinley, and K. Goldberg, “Learning ambidextrous robot grasping policies,” *Science Robotics*, vol. 4, no. 26, 2019.
- [8] A. Mousavian, C. Eppner, and D. Fox, “6-dof graspnet: Variational grasp generation for object manipulation,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019, pp. 2901–2910.
- [9] H.-S. Fang, C. Wang, M. Gou, and C. Lu, “Graspnet-1billion: A large-scale benchmark for general object grasping,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020.
- [10] M. Gou, H.-S. Fang, Z. Zhu, S. Xu, C. Wang, and C. Lu, “Rgb matters: Learning 7-dof grasp poses on monocular rgbd images,” in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 13 459–13 466.
- [11] C. Wang, H.-S. Fang, M. Gou, H. Fang, J. Gao, and C. Lu, “Graspnet discovery in clutters for fast and accurate grasp detection,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 15 964–15 973.
- [12] A. Bicchi and V. Kumar, “Robotic grasping and contact: A review,” in *Proceedings 2000 ICRA. Millennium conference. IEEE international conference on robotics and automation. Symposia proceedings (Cat. No. 00CH37065)*, vol. 1. IEEE, 2000, pp. 348–353.
- [13] K. B. Shimoga, “Robot grasp synthesis algorithms: A survey,” *The International Journal of Robotics Research*, vol. 15, no. 3, pp. 230–266, 1996.
- [14] A. T. Miller, S. Knoop, H. I. Christensen, and P. K. Allen, “Automatic grasp planning using shape primitives,” in *2003 IEEE International Conference on Robotics and Automation (Cat. No. 03CH37422)*, vol. 2. IEEE, 2003, pp. 1824–1829.
- [15] D. L. Bowers and R. Lumia, “Manipulation of unmodeled objects using intelligent grasping schemes,” *IEEE Transactions on Fuzzy Systems*, vol. 11, no. 3, pp. 320–330, 2003.
- [16] A. Saxena, J. Driemeyer, J. Kearns, and A. Ng, “Robotic grasping of novel objects,” *Advances in neural information processing systems*, vol. 19, 2006.
- [17] A. Saxena, J. Driemeyer, and A. Y. Ng, “Robotic grasping of novel objects using vision,” *The International Journal of Robotics Research*, vol. 27, no. 2, pp. 157–173, 2008.
- [18] A. Saxena, L. L. Wong, and A. Y. Ng, “Learning grasp strategies with partial shape information,” in *AAAI*, vol. 3, no. 2, 2008, pp. 1491–1494.
- [19] Q. V. Le, D. Kamm, A. F. Kara, and A. Y. Ng, “Learning to grasp objects with multiple contact points,” in *2010 IEEE International Conference on Robotics and Automation*. IEEE, 2010, pp. 5062–5069.
- [20] Y. Jiang, S. Moseson, and A. Saxena, “Efficient grasping from rgbd images: Learning using a new rectangle representation,” in *2011 IEEE International conference on robotics and automation*. IEEE, 2011, pp. 3304–3311.
- [21] I. Lenz, H. Lee, and A. Saxena, “Deep learning for detecting robotic grasps,” *The International Journal of Robotics Research*, vol. 34, no. 4-5, pp. 705–724, 2015.
- [22] J. Mahler, F. T. Pokorny, B. Hou, M. Roderick, M. Laskey, M. Aubry, K. Kohlhoff, T. Kröger, J. Kuffner, and K. Goldberg, “Dex-net 1.0: A cloud-based network of 3d objects for robust grasp planning using a multi-armed bandit model with correlated rewards,” in *2016 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2016, pp. 1957–1964.
- [23] D. Morrison, P. Corke, and J. Leitner, “Closing the loop for robotic grasping: A real-time, generative grasp synthesis approach,” *arXiv preprint arXiv:1804.05172*, 2018.
- [24] Y. Qin, R. Chen, H. Zhu, M. Song, J. Xu, and H. Su, “S4g: Amodal single-view single-shot se (3) grasp detection in cluttered scenes,” in *Conference on robot learning*. PMLR, 2020, pp. 53–65.
- [25] B. Zhao, H. Zhang, X. Lan, H. Wang, Z. Tian, and N. Zheng, “Regnet: Region-based grasp network for end-to-end grasp detection in point clouds,” in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 13 474–13 480.
- [26] Z. Dong, H. Tian, X. Bao, Y. Yan, and F. Chen, “Graspsvdn: scene-oriented grasp estimation by learning vector representations of grasps,” *Complex & Intelligent Systems*, vol. 8, no. 4, pp. 2911–2922, 2022.
- [27] J. Ichnowski, Y. Avigal, J. Kerr, and K. Goldberg, “Dex-nerf: Using a neural radiance field to grasp transparent objects,” *arXiv preprint arXiv:2110.14217*, 2021.
- [28] R. Newbury, M. Gu, L. Chumbley, A. Mousavian, C. Eppner, J. Leitner, J. Bohg, A. Morales, T. Asfour, D. Kragic, D. Fox, and A. Cosgun, “Deep learning approaches to grasp synthesis: A review,” 2022.
- [29] S. Kim, A. Shukla, and A. Billard, “Catching objects in flight,” *IEEE Transactions on Robotics*, vol. 30, no. 5, pp. 1049–1065, 2014.

- [30] A. Menon, B. Cohen, and M. Likhachev, "Motion planning for smooth pickup of moving objects," in *2014 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2014, pp. 453–460.
- [31] I. Akinola, J. Xu, S. Song, and P. K. Allen, "Dynamic grasping with reachability and motion awareness," in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2021, pp. 9422–9429.
- [32] N. Marturi, M. Kopicki, A. Rastegarpanah, V. Rajasekaran, M. Adjigble, R. Stolkin, A. Leonardis, and Y. Bekiroglu, "Dynamic grasp and trajectory planning for moving objects," *Autonomous Robots*, vol. 43, no. 5, pp. 1241–1256, 2019.
- [33] W. Yang, C. Paxton, A. Mousavian, Y.-W. Chao, M. Cakmak, and D. Fox, "Reactive human-to-robot handovers of arbitrary objects," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 3118–3124.
- [34] S. Levine, P. Pastor, A. Krizhevsky, J. Ibarz, and D. Quillen, "Learning hand-eye coordination for robotic grasping with deep learning and large-scale data collection," *The International journal of robotics research*, vol. 37, no. 4-5, pp. 421–436, 2018.
- [35] U. Viereck, A. Pas, K. Saenko, and R. Platt, "Learning a visuomotor controller for real world robotic grasping using simulated depth images," in *Conference on robot learning*. PMLR, 2017, pp. 291–300.
- [36] K. Bousmalis, A. Irpan, P. Wohlhart, Y. Bai, M. Kelcey, M. Kalakrishnan, L. Downs, J. Ibarz, P. Pastor, K. Konolige *et al.*, "Using simulation and domain adaptation to improve efficiency of deep robotic grasping," in *2018 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2018, pp. 4243–4250.
- [37] S. Song, A. Zeng, J. Lee, and T. Funkhouser, "Grasping in the wild: Learning 6dof closed-loop grasping from low-cost demonstrations," *IEEE Robotics and Automation Letters*, vol. 5, no. 3, pp. 4978–4985, 2020.
- [38] L. Wang, Y. Xiang, W. Yang, A. Mousavian, and D. Fox, "Goal-auxiliary actor-critic for 6d robotic grasping with point clouds," in *Conference on Robot Learning*. PMLR, 2022, pp. 70–80.
- [39] S. Ross, G. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proceedings of the fourteenth international conference on artificial intelligence and statistics*. JMLR Workshop and Conference Proceedings, 2011, pp. 627–635.
- [40] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain randomization for transferring deep neural networks from simulation to the real world," in *2017 IEEE/RSJ international conference on intelligent robots and systems (IROS)*. IEEE, 2017, pp. 23–30.
- [41] H. Deubel and W. X. Schneider, "Attentional selection in sequential manual movements, movements around an obstacle and in grasping," in *Attention in action*. Psychology Press, 2004, pp. 85–108.
- [42] P. Khosla, P. Teterwak, C. Wang, A. Sarna, Y. Tian, P. Isola, A. Maschinot, C. Liu, and D. Krishnan, "Supervised contrastive learning," *Advances in Neural Information Processing Systems*, vol. 33, pp. 18 661–18 673, 2020.
- [43] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.
- [44] L.-C. Chen, G. Papandreou, I. Kokkinos, K. Murphy, and A. L. Yuille, "Deeplab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected crfs," *IEEE transactions on pattern analysis and machine intelligence*, vol. 40, no. 4, pp. 834–848, 2017.
- [45] M. Guo, D. V. Gealy, J. Liang, J. Mahler, A. Goncalves, S. McKinley, J. A. Ojea, and K. Goldberg, "Design of parallel-jaw gripper tip surfaces for robust grasping," in *2017 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2017, pp. 2831–2838.
- [46] D. Morrison, P. Corke, and J. Leitner, "Egad! an evolved grasping analysis dataset for diversity and reproducibility in robotic manipulation," *IEEE Robotics and Automation Letters*, vol. 5, no. 3, pp. 4368–4375, 2020.
- [47] M. Matl *et al.*, "Pyrender," 2019.
- [48] A. Depierre, E. Dellandréa, and L. Chen, "Jacquard: A large scale dataset for robotic grasp detection," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2018, pp. 3511–3516.
- [49] C. Eppner, A. Mousavian, and D. Fox, "Acronym: A large-scale grasp dataset based on simulation," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 6222–6227.
- [50] B. Wen and K. Bekris, "Bundletrack: 6d pose tracking for novel objects without instance or category-level 3d models," in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2021, pp. 8067–8074.
- [51] S. Sajjan, M. Moore, M. Pan, G. Nagaraja, J. Lee, A. Zeng, and S. Song, "Clear grasp: 3d shape estimation of transparent objects for manipulation," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2020, pp. 3634–3642.
- [52] H. Fang, H.-S. Fang, S. Xu, and C. Lu, "Transcg: A large-scale real-world dataset for transparent object depth completion and grasping," *arXiv preprint arXiv:2202.08471*, 2022.